Composition and configuration in continuous landscape surfaces

Contents

What the package does 1

Simulated landscapes 1

The tests 2

Classifying metrics 3

Binning protocols 4

Decomposing change between two dates 5

What the package does

A continuous landscape metric computed in a landscape window can respond to the values present in the window (composition), to how those values are arranged (configuration), or to both. `surfscape` provides

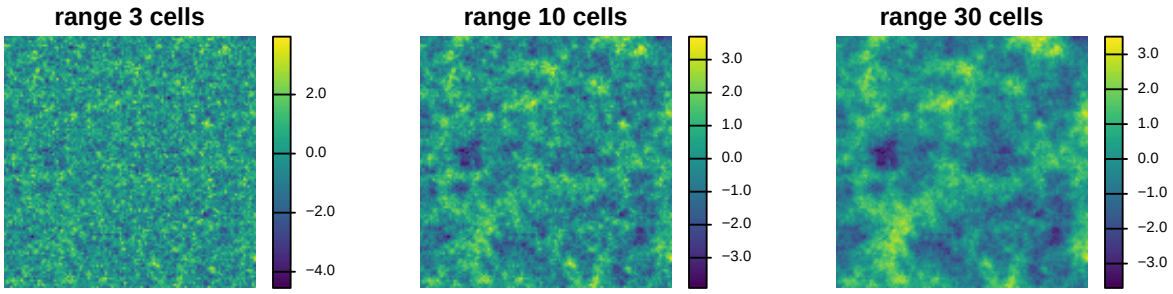
- `ss_benchmark()`: a battery of controlled tests that classifies any metric, built-in or from another package, as compositional, configurational or mixed;
- `ss_perturb()` and `ss_simulate()`: the controlled changes and simulated landscapes with known spatial structure used by the tests;
- `ss_decompose()` and `ss_decompose_landscape()`: an exact split of the change in a metric between two dates into compositional and configurational parts.

```
library(terra)
#> terra 1.9.50
library(surfscape)
```

Simulated landscapes

`ss_simulate()` draws a stationary Gaussian random field by circulant embedding. The practical range sets the distance at which the correlation between two cells falls to 0.05.

```
z3  <- ss_simulate(120, 120, range = 3,  seed = 1)
z10 <- ss_simulate(120, 120, range = 10, seed = 1)
z30 <- ss_simulate(120, 120, range = 30, seed = 1)
par(mfrow = c(1, 3), mar = c(1, 1, 2, 3))
plot(z3,  axes = FALSE, main = "range 3 cells")
plot(z10, axes = FALSE, main = "range 10 cells")
plot(z30, axes = FALSE, main = "range 30 cells")
```



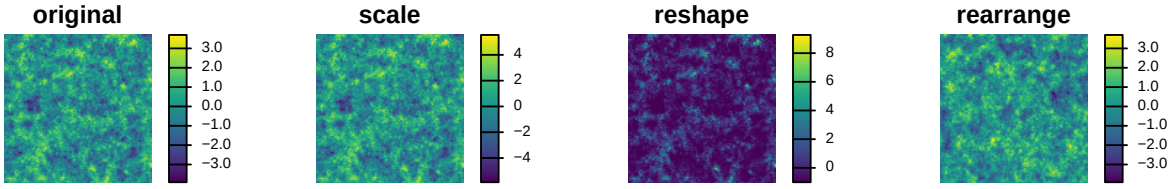
The tests

Each test changes either the values or their arrangement:

test	values present	arrangement
`permute` (inside each window)	kept	destroyed
`shift`	level changed	kept
`scale`	spread changed	kept
`reshape`	shape changed (rank preserving)	kept
`target`	replaced by another surface's distribution	kept
`rearrange` (whole landscape)	kept for the landscape	changed

`ss_perturb()` applies them to a whole surface:

```
par(mfrow = c(1, 4), mar = c(1, 1, 2, 3))
plot(z10, axes = FALSE, main = "original")
plot(ss_perturb(z10, "scale", 1.5), axes = FALSE, main = "scale")
plot(ss_perturb(z10, "reshape", 2), axes = FALSE, main = "reshape")
plot(ss_perturb(z10, "rearrange", sim_range = 10, seed = 2), axes = FALSE,
     main = "rearrange")
```

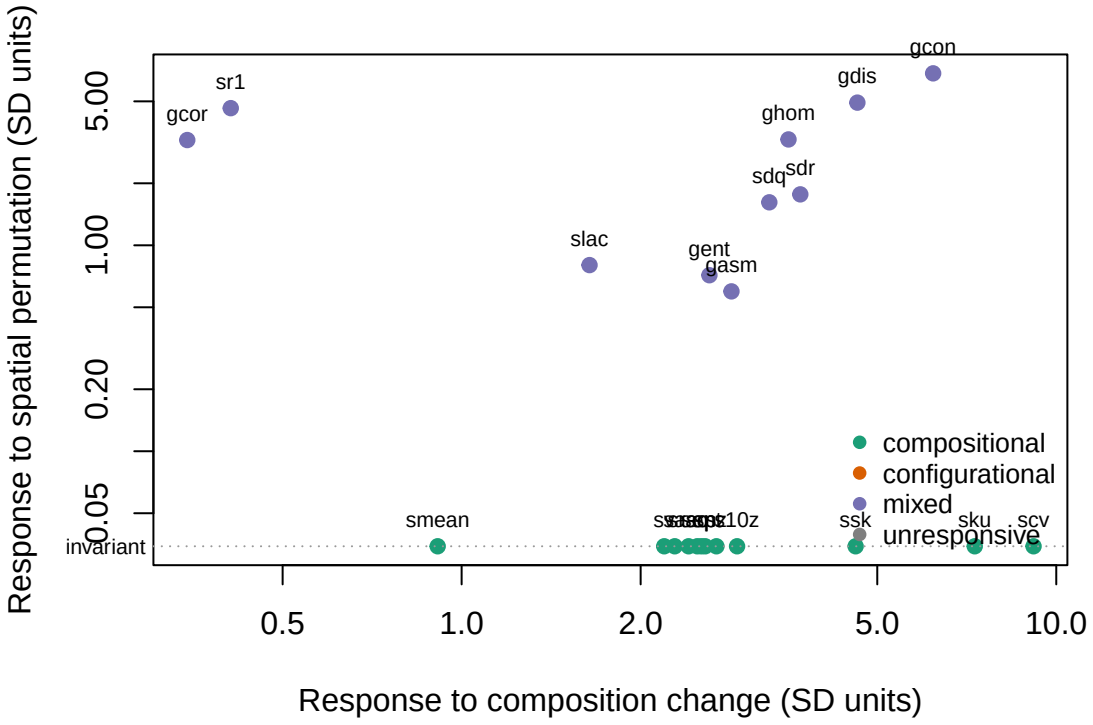


Classifying metrics

`ss_benchmark()` samples windows, applies the tests and records, for every metric and test, whether the metric was invariant (no window changed by more than a numerical tolerance), the standardised response (mean absolute change divided by the standard deviation of the metric among windows), and, for the permutation test, the proportion of windows in which the observed arrangement differs significantly from random arrangements.

```
b <- ss_benchmark(z10, w = 9, n = 60, nperm = 39,
                  tests = c("permute", "shift", "scale", "reshape"),
                  seed = 1)
b$summary[, c("metric", "category", "configuration_response",
              "composition_response", "signif")]
#>    metric      category configuration_response composition_response    signif
#> 1   smean compositional              0.0000000            0.9114396 0.0000000
#> 2      sa compositional              0.0000000            2.2822442 0.0000000
#> 3      sq compositional              0.0000000            2.4905222 0.0000000
#> 4     ssk compositional              0.0000000            4.5979405 0.0000000
#> 5     sku compositional              0.0000000            7.2926309 0.0000000
#> 6      sp compositional              0.0000000            2.5680385 0.0000000
#> 7      sv compositional              0.0000000            2.1912711 0.0000000
#> 8      sz compositional              0.0000000            2.6819123 0.0000000
#> 9    s10z compositional              0.0000000            2.9054743 0.0000000
#> 10    scv compositional              0.0000000            9.1574919 0.0000000
#> 11   sent compositional              0.0000000            2.5310286 0.0000000
#> 12   srao compositional              0.0000000            2.4080903 0.0000000
#> 13    sr1         mixed              4.6330356            0.4090081 1.0000000
#> 14    sdq         mixed              1.6167118            3.2917677 0.6833333
#> 15    sdr         mixed              1.7670269            3.7113603 0.6333333
#> 16   slac         mixed              0.8013731            1.6408970 0.9500000
#> 17   gcon         mixed              6.8364065            6.2075767 1.0000000
#> 18   gdis         mixed              4.9284841            4.6294662 1.0000000
#> 19   ghom         mixed              3.2657587            3.5454001 0.9500000
#> 20   gasm         mixed              0.5973110            2.8428445 0.6833333
#> 21   gent         mixed              0.7155411            2.6101966 0.8666667
#> 22   gcor         mixed              3.2433558            0.3456188 0.9833333
```

```
plot(b)
```



A metric is **compositional** when it is invariant to the permutation, which holds for every metric computed from the set of values alone (moments, quantiles, entropy). It is **configurational** when it responds to the permutation and is invariant to every composition test, and **mixed** when it responds to both. Lag-1 autocorrelation (`sr1`), for example, is invariant to the shift and the stretch but responds to a change in distribution shape.

Any metric function can be tested on the same windows. Here Moran's I from `terra`, which needs the window as a raster, on the first 20 windows (calling a raster function for every permuted window takes longer than the built-in kernel):

```
moran <- function(r) c(moran = unname(terra::autocor(r, method = "moran",
                                                     global = TRUE)))
ss_benchmark(z10, fun = moran, fun_input = "raster", w = 9, nperm = 39,
             tests = c("permute", "shift", "scale", "reshape"),
             cells = b$cells[1:20], seed = 1)$summary
#>   metric category configuration_response composition_response signif
#> 1  moran    mixed                3.279774             0.3573087      1
#>   permute_invariant shift_invariant scale_invariant reshape_invariant
#> 1             FALSE            TRUE            TRUE              FALSE
```

Binning protocols

Metrics that bin values (entropy, Rao's Q, lacunarity, grey-level co-occurrence textures) respond to composition partly through the binning range. With `protocol = "fixed"` both surfaces of a comparison are binned on their joint range, as required for comparisons between dates; with `protocol = "own"` each surface uses its own range.

```
m <- c("sent", "srao", "gcon", "slac")
fixed <- ss_benchmark(z10, metrics = m, w = 9, n = 60, nperm = 39,
                      tests = c("permute", "shift", "scale"), seed = 1)
own <- ss_benchmark(z10, metrics = m, w = 9, n = 60, nperm = 39,
                    tests = c("permute", "shift", "scale"), protocol = "own",
                    seed = 1)
data.frame(metric = m,
           fixed_shift = fixed$summary$shift_invariant,
           own_shift = own$summary$shift_invariant,
           own_scale = own$summary$scale_invariant)
#>   metric fixed_shift own_shift own_scale
#> 1   sent       FALSE      TRUE      TRUE
#> 2   srao       FALSE      TRUE     FALSE
#> 3   gcon       FALSE      TRUE      TRUE
#> 4   slac       FALSE      TRUE      TRUE
```

Decomposing change between two dates

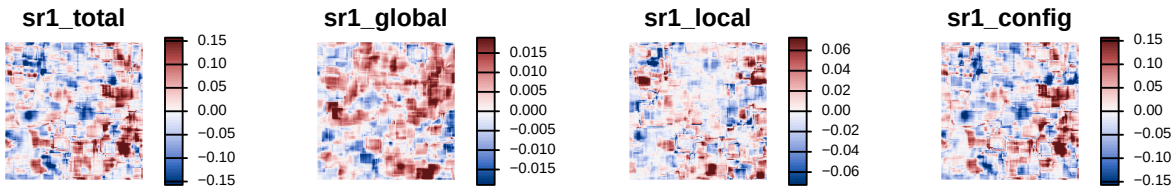
For a metric `M` computed in a window, let `W1` and `W2` be the window at the two dates. Rank transfer (`ss_rank_transfer()`) builds two artificial windows: `A`, with the spatial ordering of `W2` and the values of `W1`, and `B`, with the ordering of `W1` and the values of `W2`. Averaging over both orderings of the dates gives

- configurational component: `0.5 * [(M(A) - M(W1)) + (M(W2) - M(B))]`
- marginal (compositional) component: `0.5 * [(M(W2) - M(A)) + (M(B) - M(W1))]`

which add up to `M(W2) - M(W1)`. The same construction applied to the whole surfaces, with `Z21` (ordering of date 2, values of date 1) and `Z12`, gives the global compositional component of each window, `0.5 * [(M(Z2) - M(Z21)) + (M(Z12) - M(Z1))]`, and the local compositional component is the marginal component minus the global one. If the second date is a strictly increasing transformation of the first, the configurational and local components are zero; if both dates contain the same values, the global component is zero.

```
z1 <- rast(system.file("extdata", "ndvi_2013.tif", package = "surfscape"))
z2 <- rast(system.file("extdata", "ndvi_2023.tif", package = "surfscape"))
d <- ss_decompose(z1, z2, metrics = c("sq", "sr1"), w = 15)
v <- values(d[[c("sr1_total", "sr1_global", "sr1_local", "sr1_config")]])
max(abs(v[, 1] - rowSums(v[, 2:4])), na.rm = TRUE)   # additivity
#> [1] 0
range(values(d[["sq_config"]]), na.rm = TRUE)        # zero for Sq
#> [1] -5.551115e-17  5.551115e-17
```

```
par(mfrow = c(1, 4), mar = c(1, 1, 2, 3))
for (k in c("sr1_total", "sr1_global", "sr1_local", "sr1_config")) {
  r <- d[[k]]
  mx <- max(abs(quantile(values(r), c(0.02, 0.98), na.rm = TRUE)))
  plot(clamp(r, -mx, mx, values = TRUE), axes = FALSE, main = k,
       range = c(-mx, mx) * 1.001, col = hcl.colors(100, "Blue-Red 3"))
}
```



At the landscape level the window is the whole surface:

```
ss_decompose_landscape(z1, z2, metrics = c("sq", "ssk", "sent", "sr1", "gcon"))
#>   metric    value1     value2        total  composition configuration
#> 1     sq 0.0466841 0.05185955  0.005175444  0.005175444 -2.428613e-17
#> 2    ssk 2.6089163 2.37382818 -0.235088152 -0.235088152 -4.662937e-15
#> 3   sent 1.8935259 2.00218632  0.108660435  0.108660435  0.000000e+00
#> 4    sr1 0.8527778 0.84894064 -0.003837182  0.006750241 -1.058742e-02
#> 5   gcon 0.8005063 0.96190224  0.161395952  0.113285727  4.811023e-02
#>   share_config
#> 1 4.692569e-15
#> 2 1.983484e-14
#> 3 0.000000e+00
#> 4 6.106603e-01
#> 5 2.980882e-01
```

and `ss_scale_profile()` repeats the decomposition over window sizes:

```
ss_scale_profile(z1, z2, metrics = c("sq", "sr1"), w_seq = c(5, 15, 31),
                 progress = FALSE)[, c("w", "metric", "share_config",
                                       "r_comp_config", "sign_disagree")]
#>    w metric share_config r_comp_config sign_disagree
#> 1  5     sq 1.297708e-16  -0.009697247            NA
#> 2  5    sr1 7.235036e-01  -0.050396078     0.1063619
#> 3 15     sq 4.965177e-16   0.002872331            NA
#> 4 15    sr1 6.604718e-01  -0.200500097     0.1364609
#> 5 31     sq 1.243938e-15  -0.003851532            NA
#> 6 31    sr1 6.204474e-01  -0.277951576     0.1421453
```

Decomposed change can be summarised around field sites with `ss_extract()`:

```
pts <- read.csv(system.file("extdata", "sites.csv", package = "surfscape"))
head(ss_extract(d, pts, radii = c(150, 300)))
#>   site radius    layer         value
#> 1    1    150 sq_total  0.0193376385
#> 2    2    150 sq_total -0.0002010769
#> 3    3    150 sq_total  0.0066282079
#> 4    4    150 sq_total  0.0090591968
#> 5    5    150 sq_total  0.0049731541
#> 6    6    150 sq_total  0.0077608692
```